



**KARADENİZ TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF SOFTWARE ENGINEERING**

GRAMMARGRAPH: DYNAMIC AST GENERATOR

Student Names: Elanur Kurt, Eylül Kalfa, İrem Su Köse
Student Numbers: 439199, 439184, 439183

Date: May 10, 2026

Contents

1	Introduction	2
2	Project Objectives	3
3	Aims and Objectives	3
4	Methodology	4
4.1	Grammar Rules Representation	4
4.2	Abstract Syntax Tree Output	4
4.3	Recursive Grammar Parsing	4
4.4	System Modules	4
5	Findings	6
5.1	Sample Parsing Results	6
5.2	Generated Parse Tree Visualization	6
6	Conclusion and Discussion	7

Abstract

This project presents the implementation of a custom Syntax Analyzer and Parse Tree Visualizer developed in Python for analyzing Context-Free Grammars (CFGs) written in Backus-Naur Form (BNF). The system reads grammar rules from external text files, tokenizes input sentences, validates whether the token sequences conform to the given grammar, and generates corresponding parse trees. The parser is implemented using the Recursive Descent Parsing technique, which follows a top-down parsing strategy.

The project supports both standard English sentence structures and grammars containing epsilon (ϵ) productions. Additionally, the system provides detailed error reporting mechanisms for invalid sentences, including token position, expected grammar rules, and descriptive rejection messages. Valid parse results are represented in multiple formats, including hierarchical console output, nested JSON structures, and graphical Abstract Syntax Tree (AST) visualizations generated with Graphviz and pydot libraries.

The implementation demonstrates practical applications of formal language theory, recursive parsing algorithms, compiler design concepts, and data structure usage in syntax analysis systems.

Keywords

Context-Free Grammar, Recursive Descent Parser, Syntax Analyzer, Parse Tree, Abstract Syntax Tree, BNF, Python, Compiler Design, Graphviz, JSON

1 Introduction

Programming languages rely heavily on syntax analysis to determine whether a sequence of tokens conforms to a predefined grammar. Syntax analyzers, also known as parsers, are essential components of compilers, interpreters, and language-processing systems. They validate input structures and generate internal representations such as parse trees or abstract syntax trees.

The purpose of this project is to develop a custom Syntax Analyzer and Parse Tree Visualizer using Python. The system is designed to read Context-Free Grammar (CFG) rules written in Backus-Naur Form (BNF) from external text files and analyze whether given sentences conform to these rules.

If the input sentence is syntactically valid, the parser generates:

- A hierarchical text-based parse tree,
- A nested JSON representation,
- A graphical Abstract Syntax Tree (AST) visualization in PNG format.

If the sentence is invalid, the system provides detailed and user-friendly error messages instead of terminating unexpectedly.

The project focuses on several important concepts in programming language theory and compiler construction:

- Context-Free Grammars (CFG),
- Recursive Descent Parsing,

- Abstract Syntax Trees,
- Recursive algorithms,
- Error handling mechanisms,
- Data structure design.

The implementation was developed entirely in Python and uses external libraries such as Graphviz and pydot for visualization purposes.

2 Project Objectives

The main objectives of this project are:

1. To design a syntax analyzer capable of reading BNF grammar rules dynamically from text files.
2. To implement a Recursive Descent Parser using top-down parsing techniques.
3. To validate whether token sequences conform to the specified grammar.
4. To support grammars containing epsilon (ϵ) productions.
5. To generate parse trees both textually and graphically.
6. To produce machine-readable JSON representations of parse trees.
7. To implement detailed error handling mechanisms for invalid inputs.
8. To demonstrate the practical use of formal language theory and compiler construction concepts.

3 Aims and Objectives

The primary aim of this project is to develop a custom Syntax Analyzer and Parse Tree Visualizer using Python for processing Context-Free Grammars (CFGs) written in Backus-Naur Form (BNF).

The objectives of the project are:

- To read grammar rules dynamically from external text files,
- To implement a Recursive Descent Parser for syntax analysis,
- To validate whether input sentences conform to the given grammar,
- To generate parse trees in textual, JSON, and graphical formats,
- To support grammars containing epsilon (ϵ) productions,
- To provide detailed and user-friendly error handling for invalid inputs.

4 Methodology

This section describes the methods and procedures used in our project. For the SE2002 Programming Languages course, we developed a Recursive Descent Parser in Python to meet the "Expression Evaluation and Selector Design" requirements. During the development process, we did not follow a strict division of labor or a sequential model. Instead, our team consisting of Elanur Kurt, İrem Su Köse, and Eylül Kalfa came together at all stages to brainstorm, and we completed the algorithm design and coding processes by working collaboratively.

The system consists of a modular structure that can dynamically read grammar rules from an external file, check whether the given sentences or word sequences comply with these rules, and generate an Abstract Syntax Tree (AST) if they are appropriate.

4.1 Grammar Rules Representation

Our parser processes structured grammar rules as terminal and non-terminal expressions. Table 1 shows some of the basic word types and terminal options recognized by the system.

Non-Terminal Expression	Terminal Options
<determiner>	the, a
<pronoun>	he, she, it, they, me, you
<adjective>	big, small, happy, lazy, red
<verb>	saw, liked, admired
<conjunction>	and, or
<adverb>	quickly,slowly,carefully,loudly
<noun>	cat,dog,man,telescope

Table 1: Sample Grammar Terminals

4.2 Abstract Syntax Tree Output

Below is an example of the tree output generated by the system:

4.3 Recursive Grammar Parsing

Our system can successfully parse recursive rules read from grammar files. For example, the production function of rule A can be mathematically expressed as:

$$A \rightarrow aA \mid \epsilon$$

Here, a represents a terminal, and A represents a non-terminal that can recursively call itself. The empty string condition is denoted by ϵ .

4.4 System Modules

Our project is shaped around three main modules in accordance with Object-Oriented Programming (OOP) principles:

```

=====
Input: the cat and dog saw a man
Status: Valid

PARSE TREE:
-> Görsel başarıyla oluşturuldu: tree_output_1_sentence_1.png

JSON REPRESENTATION:
{
  "sentence": {
    "noun-phrase": {
      "determiner": "the",
      "adjective-list": {
        "ε": "empty"
      },
      "noun-list": {
        "noun": "dog",
        "conjunction": "and"
      }
    },
    "verb-phrase": {
      "verb": "saw",
      "noun-phrase": {
        "determiner": "a",
        "adjective-list": {
          "ε": "empty"
        },
        "noun-list": {
          "noun": "man"
        }
      },
      "adverb-list": {
        "ε": "empty"
      }
    }
  }
}

```

Figure 1: Sample Abstract Syntax Tree (AST) output generated for the sentence "the cat and dog saw a man" using the `print_tree` function in `nodes.py`.

- **grammar_reader.py**: Reads the rules from .txt files line by line, parses the left (lhs) and right (rhs) sides using the `::=` and `|` operators, and converts the system into a dictionary structure.
- **nodes.py**: Contains the `Node` class representing each element of the tree created during the parsing process. It manages child nodes and provides a hierarchical tree drawing (`print_tree`) on the terminal screen.
- **parser_logic.py**: The main parser that manages the system. It validates sentences according to grammar rules. It also includes a custom error-handling mechanism that deeply reports the missing structure ("expected") and the reason for the error ("reason") when an unexpected situation is encountered.

5 Findings

The developed syntax analyzer successfully parsed and validated multiple input sentences based on the provided Context-Free Grammar (CFG) rules. The system was tested using both standard English sentence grammars and recursive grammars containing epsilon (ϵ) productions.

The parser correctly identified valid and invalid inputs while generating detailed parse tree structures. For valid sentences, the program produced:

- Hierarchical text-based parse trees,
- Nested JSON representations,
- Graphical parse tree visualizations in PNG format.

Additionally, the error handling mechanism successfully detected syntax mismatches and provided descriptive error messages indicating the exact error position and expected grammar rule.

5.1 Sample Parsing Results

Input Sentence	Result
the big dog runs quickly	Invalid
the man liked the dog	Valid
the quickly dog	Invalid
aaabbb	Valid
abbbba	Invalid

Table 2: Sample Parsing Results

5.2 Generated Parse Tree Visualization

Figure 2 presents an example parse tree generated by the system using the Graphviz and pydot libraries.

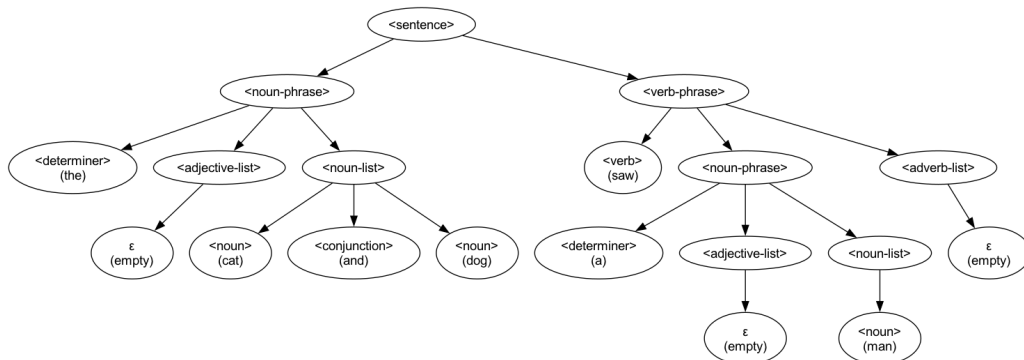


Figure 2: Sample Parse Tree Visualization

The findings demonstrate that the Recursive Descent Parser successfully performs syntax analysis, tree generation, and visualization while maintaining modularity and clear error reporting.

6 Conclusion and Discussion

This project successfully demonstrates the implementation of a custom Syntax Analyzer and Parse Tree Visualizer using Python. By utilizing Recursive Descent Parsing techniques, the system effectively validates Context-Free Grammar structures written in BNF notation and generates multiple forms of parse tree representations.

The project combines theoretical concepts from compiler construction and formal language theory with practical software engineering implementation techniques. Support for epsilon productions, graphical AST generation, and detailed error handling mechanisms significantly improve the functionality and usability of the system.

The modular design enables future extensibility and adaptation for more advanced compiler-related applications. Overall, the project provides valuable practical experience in parsing algorithms, recursive programming, data structure management, and syntax analysis systems.

References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd Edition, Pearson Education, 2006.
2. Hopcroft, J. E., Motwani, R., & Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd Edition, Pearson, 2006.
3. Python Software Foundation, “Python Documentation.” Available: <https://docs.python.org/3/>
4. Graphviz Documentation, “Graph Visualization Software.” Available: <https://graphviz.org/>
5. pydot Documentation, “Python Interface to Graphviz.” Available: <https://github.com/pydot/pydot>
6. GeeksforGeeks, “Recursive Descent Parser.” Available: <https://www.geeksforgeeks.org/recursive-descent-parser/>